5.1. Introduction to Level 1 View

This view provides a high-level overview of the logical structure of the Teaching Vacancies Python/Django application. It identifies the main building blocks, which are organized as distinct Django applications (apps). Each app encapsulates a specific domain or set of functionalities, promoting modularity and separation of concerns. The diagram below illustrates these apps and their primary dependencies on one another.

5.2. Django App Descriptions

The system is composed of the following key Django apps:

- `users`
- **Responsibilities:** Manages user authentication, user accounts, and different user roles (Jobseeker, Publisher, Support User). It defines the custom User model that other apps will reference.
- **Key Models:** `User`
- `core`
- **Responsibilities:** Contains shared functionality, base models (if any, though often abstract models are in specific apps or a `common` app), core utilities, custom middleware, and general site-wide elements that don't fit into a specific domain app. This also includes models like `Feedback` that can relate to multiple other apps.
- **Key Models:** `Feedback` (and potentially others like `LocationPolygon` if not in `organisations`)
- `organisations`
- **Responsibilities:** Manages all aspects of organisations, including schools, school groups (Trusts, Local Authorities). This includes their profiles, addresses, contact details, and relationships between them (e.g., schools within a group).
- **Key Models:** `Organisation`, `SchoolGroupMembership` (if not handled by M2M on Organisation itself)
- `vacancies`
- **Responsibilities:** Manages the creation, display, and lifecycle of job vacancies. This includes all details related to a job posting, such as job title, description, salary, working patterns, subjects, application methods, and status.
- **Key Models:** `Vacancy`, `OrganisationVacancy` (through model), `SupportingDocument` (if used)

- `jobseekers`
- **Responsibilities:** Manages jobseeker-specific data and functionalities beyond basic user authentication. This includes jobseeker profiles, job preferences (roles, locations, subjects), saved jobs, and job alert subscriptions.
- **Key Models:** `JobseekerProfile`, `PersonalDetails`, `JobPreference`, `JobPreferenceLocation`, `SavedJob`, `Subscription`, `AlertRun`
- `job_applications`
- **Responsibilities:** Manages the process of jobseekers applying for vacancies. This includes storing application details, tracking application status, and handling data related to qualifications, employment history, references, and personal statements for each application.
- **Key Models:** `JobApplication`, `Employment`, `Qualification`, `QualificationResult`, `Referee`, `TrainingAndCpd`, `ProfessionalBodyMembership` (these models are also used by `jobseeker_profiles` but are often created/managed in the context of an application)
- `publishers`
- **Responsibilities:** Manages publisher-specific data and functionalities beyond basic user authentication and organisation management. This could include publisher preferences, links to specific vacancies they've created, and potentially models related to ATS integration if not in `api`.
- **Key Models:** `PublisherPreference`, `PublisherAtsApiClient` (if not in `api`)
- `notifications`
- **Responsibilities:** Manages the generation and sending of notifications to users (e.g., job alerts, application confirmations, new feature announcements). This app would integrate with GOV.UK Notify. It might also store a record of notifications sent.
- **Key Models:** `NotificationRecord` (or similar, if tracking sent notifications beyond what Notify provides)
- `api`
- **Responsibilities:** Provides external API endpoints for third-party integrations, such as allowing external Applicant Tracking Systems (ATS) to post vacancies. It may also serve internal frontend needs if a decoupled frontend is used in the future.
- **Key Models:** (Typically doesn't own models, but exposes data from other apps via serializers)

5.3. Building Block Diagram (Level 1)

The following diagram illustrates the main Django apps and their high-level relationships:

```
@startuml
!theme materia
skinparam componentStyle rect

package "Teaching Vacancies (Python) - Django Apps" {
  [users] <<App>>
  [core] <<App>> # General models like Feedback
  [organisations] <<App>>
  [vacancies] <<App>>
  [jobseekers] <<App>> # Jobseeker profiles, preferences, saved jobs, subscriptions
  [job_applications] <<App>> # Application process and data
  [publishers] <<App>> # Publisher-specific data like preferences
  [notifications] <<App>>
  [api] <<App>>

  ' Core User and Authentication
  jobseekers ..> users : User Profile
  publishers ..> users : User Profile
  job_applications ..> users : Applicant (Jobseeker)
  vacancies ..> users : Posted by (Publisher)
  notifications ..> users : Notifies User

  ' Vacancy Lifecycle
  vacancies ..> organisations : Vacancy at Organisation(s)
  job_applications ..> vacancies : Application for Vacancy

  ' Publisher Interactions
  publishers ..> organisations : Manages Organisation(s)
  publishers ..> vacancies : Creates/Manages Vacancies
  publishers ..> job_applications : Views Applications for their Vacancies

  ' Jobseeker Interactions
  jobseekers ..> organisations : (For location preferences, excluding orgs from profile visi
  jobseekers ..> vacancies : Saves Job, Subscribes to Alerts based on Vacancy criteria

  ' API exposing core data
  api ..> vacancies : Exposes Vacancy data
  api ..> organisations : Exposes Organisation data

  ' Core app usage (Feedback, etc.)
  core ..> users : Feedback from User
```

```
  core ..> vacancies : Feedback on Vacancy
  core ..> job_applications : Feedback on Job Application
  core ..> jobseekers : Feedback on Subscription (via jobseekers.Subscription)

  ' Implicit dependencies on 'core' for base models or utilities are not explicitly drawn
  ' but 'core' can be seen as a foundational app.
  ' Many apps might have an implicit dependency on 'users' for the User model.

  ' Layout hints to try and group related apps
  users -[hidden]r- core
  organisations -[hidden]l- core
  vacancies -[hidden]u- core
  jobseekers -[hidden]d- users
  job_applications -[hidden]r- jobseekers
  publishers -[hidden]d- users
  notifications -[hidden]r- api
}
@enduml
```

Explanation of Key Dependencies:

- **`users`:** Central for authentication; `jobseekers`, `publishers`, `job_applications`, `vacancies` (via publisher), and `notifications` all depend on it.
- **`organisations`:** Provides data for `vacancies` (where the job is located) and for `publishers` (organisations they manage). `jobseekers` might also reference it for profile preferences (e.g., excluding specific organisations).
- **`vacancies`:** Core to the system. `job_applications` are made for vacancies. `jobseekers` save and subscribe to alerts based on vacancy criteria. `publishers` create and manage vacancies. The `api` app exposes vacancy data.
- **`jobseekers`:** Relies on `users` for the base account. It interacts with `vacancies` for saved jobs/alerts and `organisations` for location/preference settings.
- **`job_applications`:** Directly links to `users` (the applicant), and `vacancies` (the job applied for). It also manages data models like `Employment` and `Qualification` which might also be linked to `JobseekerProfile`.
- **`publishers`:** Relies on `users` for the base account. Interacts heavily with `organisations` and `vacancies` for management tasks, and `job_applications` for viewing submitted applications.
- **`core`:** May contain models like `Feedback` which can link to `User`, `Vacancy`, `JobApplication`, and `Subscription` (via `jobseekers` app). Other apps might use utilities from `core`.

- **`notifications`:** Primarily depends on `users` to send notifications and potentially other apps (like `vacancies` or `job_applications`) for triggering events.
- **`api`:** Exposes data from core apps like `vacancies` and `organisations`.

This Level 1 view sets the stage for more detailed exploration of each app's internal structure and relationships in subsequent views (Level 2).